Проект под названием **Outpost** представляют собой подробное руководство по созданию современной облачной SaaS-платформы (Agentic Workspace Platform) с нуля до готового развертывания.

Проект создан блогером с канала **RoadsideCoder** для портфолио разработчиков.

Ниже подробно разобрано, что это за проект, какую проблему он решает и как всё это работает.

1. Какую проблему решает проект?

Обычно, когда вы работаете над сложной задачей в команде или с ИИ, вам приходится переключаться между множеством разных инструментов:

- В одном месте вы переписываетесь с ChatGPT или Claude.
- В другом (например, в Miro или Excalidraw) рисуете схемы и архитектуру.
- В третьем (Notion, Google Docs) пишете техническую документацию.
- В четвёртом (Trello, Jira) управляете задачами на доске (Kanban).

Outpost объединяет всё это в единое «агентское» рабочее пространство (Agentic Workspace). Вы общаетесь с искусственным интеллектом, а он не просто отвечает текстом в чате, а напрямую управляет интерфейсом: создаёт элементы на интерактивной доске и пишет структурированные документы на ваших глазах.

2. Что представляет собой Outpost на практике?

Платформа совмещает несколько ключевых интерфейсов и возможностей:

1. **Интерактивный ИИ-агент (Agentic Workflow):**
   - Вы вводите текстовый или голосовой запрос (например: *«Создай структуру базы данных и опиши её»*).
   - ИИ анализирует задачу, отправляет мысли (reasoning) в чат, после чего **в реальном времени рисует схему** на холсте (canvas) и параллельно **заполняет документ** текстом.
2. **Две основные рабочие поверхности (Surfaces):**
   - **Холст (Canvas):** Интерактивная доска на базе библиотеки *tldraw*, где ИИ или пользователь могут рисовать блоки, схемы и диаграммы.
   - **Документы (Docs):** Текстовый редактор в стиле Notion (на базе *BlockNote* и *Yjs* для совместной работы).
3. **Управление проектами (Kanban Dashboard):**
   - Все ваши рабочие пространства сохраняются в личном или командном дашборде, где их можноорганизовывать с помощью Kanban-доски (статусы: *Planning*, *In Progress*, *In Review*, *Done*).
4. **Командная работа и B2B-фичи:**
   - Поддержка создания организаций, приглашения участников, разграничения прав и биллинга (через *Clerk* и *Supabase*).

3. Из каких технологий состоит проект (Tech Stack)?

Это полноценный Full-Stack проект уровня production на современном технологическом стеке:

- **Фреймворк:** Next.js (App Router, Server Actions).
- **Язык:** TypeScript.
- **База данных и бэкенд:** Supabase (PostgreSQL + Realtime) с использованием ORM Prisma.
- **ИИ-интеграция:** Vercel AI SDK совместно с моделями Google Gemini (с поддержкой стриминга мыслей и вызова инструментов — tool calling).
- **Интерфейс (UI):** Tailwind CSS, компоненты shadcn/ui.
- **Инструменты холста и документов:** tldraw (для схем) и BlockNote (для документов).
- **Авторизация и роли:** Clerk (с поддержкой организаций).

Зачем это видео вам?

Автор записывает это видео как практический пошаговый гайд, чтобы разработчики могли пройти весь путь от написания первой строки кода до деплоя готового SaaS-сервиса на Vercel. Полученный проект можно смело добавлять в резюме, так как он демонстрирует владение самыми передовыми трендами веб-разработки 2026 года: работу с ИИ-агентами, real-time синхронизацию и сложные интерфейсы.

ТЕХНИЧЕСКИЙ ОТЧЕТ: АРХИТЕКТУРА И РЕАЛИЗАЦИЯ PLATFORM OUTPOST (FULL-STACK AGENTIC WORKSPACE)

RoadsideCoder - Full Stack React Project (Agentic Workspace Platform)

---

1. Введение и полная концепция платформы Outpost

Разработка современного программного обеспечения переживает фундаментальный сдвиг, определяемый переходом от изолированных текстовых или визуальных редакторов к интерактивным средам совместной работы с участием искусственного интеллекта. Объединение диаграмм и текстовых спецификаций в едином AI-пространстве стало ключевым трендом, поскольку системное проектирование требует непрерывной связи между визуальными схемами (ERD, архитектурные потоки) и их форматированным текстовым описанием (технические требования, API-спецификации). Платформа Outpost представляет собой полнофункциональное B2B SaaS-решение класса Full-Stack Agentic Workspace, предназначенное для устранения барьера между графическим и текстовым представлением системных моделей путем их объединения в единый контекстный контур автономного AI-агента.

Анализ фундаментальной проблемы

Традиционный инженерный процесс глубоко фрагментирован из-за структурного разрыва между визуальными архитектурными схемами и текстовой технической документацией. Основными проблемами и издержками этого разрыва являются:

1. **Рассинхронизация контекста (Context Drift):** Любое изменение структуры базы данных или архитектуры сервисов на схеме требует ручного обновления текстовой документации. В реальных условиях жестких дедлайнов ручная синхронизация уступает место приоритетам разработки, что приводит к появлению устаревших спецификаций и архитектурным ошибкам.
2. **Высокие накладные расходы на контекстное переключение (Context Switching):** Инженеры и архитекторы вынуждены постоянно переключаться между независимыми инструментами (например, Notion/Confluence для текста и Miro/Excalidraw для диаграмм), теряя фокус и целостность восприятия проектируемой системы.
3. **Ограниченность традиционных AI-помощников:** Классические диалоговые AI-чаты генерируют обособленный текст или фрагменты кода, не имея прямого доступа к объектной модели визуального холста и не способные манипулировать UI-элементами и блоками документов в реальном времени.

Декомпозиция концепции Outpost

Платформа Outpost спроектирована как автономная AI-среда, построенная с нуля на современно сбалансированном веб-стеке. Фундаментальное определение платформы формулируется следующим образом:

"Build your own Agentic Workspace App from scratch. A Full Stack AI SaaS in Next.js where an AI agent builds diagrams on a canvas (tldraw) and Notion like doc (BlockNote) using custom agent tools, with Gemini and the Vercel AI SDK, voice

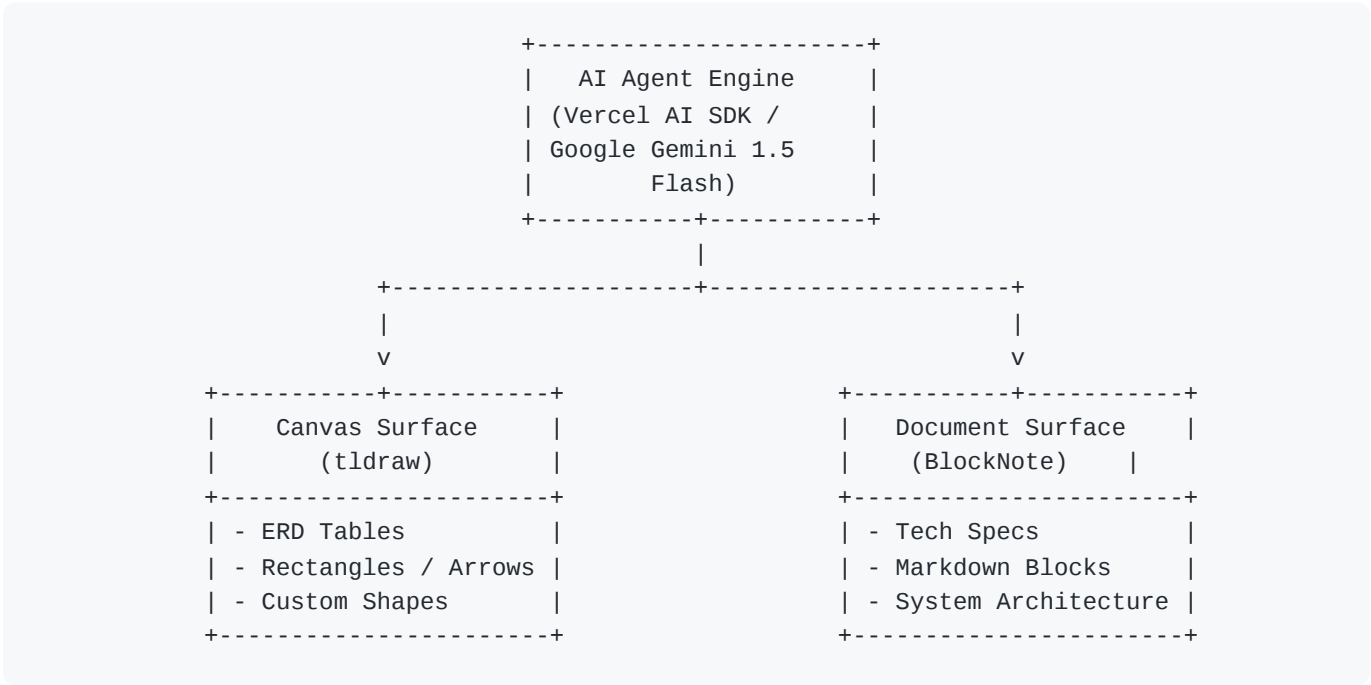
input, real-time collaboration, personal and organization workspaces, Clerk for Google authentication and subscription billing, Supabase as PostgreSQL DB with Prisma, and Shadcn UI Tutorial."

Архитектурный стек платформы состоит из пяти ключевых подсистем:

- **Двухэкранный UI-слой (Dual-Surface Workspace):** Бесконечный векторный холст на базе `@tldraw/tldraw` и блочный редактор документов на базе `@blocknote/react` / `@blocknote/core`, функционирующие бок о бок в едином React-контексте.
- **Слой оркестрации AI:** Vercel AI SDK в комбинации с моделью `Google Gemini 1.5 Flash` (поддерживающей режим рассуждений `thinkingConfig` с `includeThoughts: true` и нативный вызов функций `Tool Calling`).
- **Слой мультиарендности и монетизации:** Clerk (аутентификация Google, B2B-организации, разграничение личных и командных пространств) и Clerk Billing (Seat-Based тарификация организаций, отслеживание квот и лимитов AI-промптов).
- **Слой данных и распределенной синхронизации:** База данных PostgreSQL в Supabase, ORM Prisma и гибридный протокол реального времени на базе Yjs (CRDT) и Supabase Realtime Broadcast Channels.
- **Пользовательский интерфейс и управление:** Компоненты дизайна Shadcn UI (Nova theme / Radix UI primitives), система драг-н-дроб управления Kanban-доской (`@hello-pangea/dnd`) и подсистема голосового ввода на базе нативного браузерного Web Speech API.

Механика работы AI-агента

AI-агент в Outpost выступает активным участником рабочего процесса, работая одновременно с двумя типами поверхностей (Canvas и Doc). Взаимодействие с агентом осуществляется через текстовые инструкции или распознавание речи в реальном времени.



При получении пользовательского промпта (например, *"Разработай схему базы данных и техническую спецификацию для сервиса доставки"*), агент выполняет следующий цикл:

1. **Считывание текущего состояния:** Агент запрашивает текущие снимки состояния холста (`summarizeCanvas`) и документа (`summarizeDoc`).

2. **Отображение потока рассуждений (Live Thinking):** В реальном времени пользователь видит внутренний поток рассуждений модели (блоки `thinking`), передаваемый через Server-Sent Events.
3. **Параллельный вызов клиентских инструментов (Tool Execution):** Агент последовательно вызывает инструменты — добавляет таблицы базы данных на холст `tldraw` (кастомные фигуры `entityTable`), строит связи (`createArrowBetween`) и одновременно генерирует Markdown-блоки спецификации в `BlockNote`.

Обеспечение параллельной работы пользователя и AI-агента над едиными объектами требует создания отказоустойчивой инфраструктуры синхронизации данных, исключающей состояние гонки и потери оперативного контекста.

---

2. Детальный разбор Yjs (YGS) и инфраструктуры синхронизации

При проектировании мультиарендных B2B-систем совместной работы ключевой инженерной задачей является выбор алгоритма разрешения конфликтов (Conflict Resolution). В условиях, когда AI-агент и несколько пользователей одновременно модифицируют один и тот же документ или элементы визуального холста, классические подходы на основе блокировок ресурсов или простых меток времени (Last-Write-Wins) приводят к фатальной потере данных и блокировке пользовательского интерфейса.

Теоретические основы Yjs и CRDT

В платформе Outpost для синхронизации структуры документов используется библиотека **Yjs** — высокопроизводительная реализация конфликтно-устойчивых реплицируемых типов данных (**CRDT, Conflict-free Replicated Data Types**).

В основе CRDT Yjs лежит математическая модель структурного изменения без централизованных блокировок. Вместо отправки полных документов, клиенты обмениваются бинарными атомарными операциями (дельтой изменений). Каждый элемент данных снабжается уникальным идентификатором, состоящим из **Client ID** и **локального счетчика (Clock / Векторные часы)**.

При возникновении параллельных изменений алгоритм Yjs автоматически приводит состояние документов на всех узлах к абсолютно идентичному виду (**Strong Eventual Consistency**) без участия центрального сервера-арбитра. Это достигается благодаря свойствам коммутативности, ассоциативности и идемпотентности операций слияния векторов состояния в CRDT.

Сравнительный анализ: Yjs vs OT vs WebSocket Broadcasting

Критерий сравнения	**Yjs (CRDT)**	**Operational Transformation (OT)**	**WebSocket Broadcasting (Raw State)**
Устойчивость к офлайн-режиму	Высокая: Клиент вносит изменения локально; слияние происходит автоматически при подключении.	Низкая: Требуется постоянное подключение к центральному серверу для расчета трансформации индексов.	Отсутствует: Офлайн-изменения перезаписывают состояние других пользователей (Last-Write-Wins).

Критерий сравнения	Yjs (CRDT)	Operational Transformation (OT)	WebSocket Broadcasting (Raw State)
Децентрализованное слияние	Да: Математически гарантированная сходимость векторов состояния на клиентах.	Нет: Необходим выделенный центральный сервер-арбитр (типа Google Docs server).	Нет: Требуется ручная резолюция конфликтов на стороне сервера.
Бинарная компактность	Высокая: Обновления кодируются в формате lib0 encoding с применением дельта-сжатия.	Средняя: Зависит от текстового формата JSON-операций.	Низкая: Передается полный JSON-объект состояния при каждом изменении.
Сложность инфраструктуры	Минимальная: Работает поверх любых транспортов (WebRTC, BroadcastChannels, Supabase).	Очень высокая: Требует сложных алгоритмов преобразования операционных индексов.	Низкая: Простая трансляция JSON-сообщений через socket-сервер.

Глубокий разбор кода синхронизации и анализ состояния гонки

Инфраструктура синхронизации документов в Outpost построена на связке Yjs, BlockNote, канала Supabase Realtime Broadcast и персистентного слоя PostgreSQL.

1. Конвертация бинарных данных Base64

Для передачи бинарных дельт Yjs (`Uint8Array`) через текстовые WebSocket-каналы Supabase Realtime и их сохранения в строковых полях PostgreSQL используется модуль конвертации:

```
// lib/b64.ts
export function toB64(bytes: Uint8Array): string {
  return btoa(String.fromCharCode(...bytes));
}

export function fromB64(base64: string): Uint8Array {
  const binaryString = atob(base64);
  const bytes = new Uint8Array(binaryString.length);
  for (let i = 0; i < binaryString.length; i++) {
    bytes[i] = binaryString.charCodeAt(i);
  }
  return bytes;
}
```

2. Кастомный провайдер синхронизации `SupabaseYJSProvider`

Провайдер организует подключение к эфирному каналу вещания Supabase Realtime (Broadcast channels). Он управляет событиями: `update` (дельты документа), `awareness` (курсоры и присутствие пользователей) и

`sync-request` (запрос полного состояния новым клиентом).

```
// lib/supabase-yjs-provider.ts
import * as Y from 'yjs';
import { Awareness, applyAwarenessUpdate, encodeAwarenessUpdate } from 'y-
protocols/awareness';
import { RealtimeChannel } from '@supabase/supabase-js';
import { supabase } from '@/lib/supabase';
import { fromB64, toB64 } from '@/lib/b64';

export class SupabaseYJSProvider {
  public awareness: Awareness;
  private channel: RealtimeChannel;
  private isSubscribed: boolean = false;

  constructor(
    private doc: Y.Doc,
    private roomID: string,
    onReady?: () => void
  ) {
    this.awareness = new Awareness(this.doc);
    this.channel = supabase.channel(`doc:${this.roomID}`, {
      config: { broadcast: { self: false } },
    });

    // Прием дельт обновлений Yjs от других пиров
    this.channel.on('broadcast', { event: 'update' }, ({ payload }) => {
      const update = fromB64(payload.update);
      Y.applyUpdate(this.doc, update, this);
    });

    // Синхронизация присутствия и курсоров пользователей (Awareness)
    this.channel.on('broadcast', { event: 'awareness' }, ({ payload }) => {
      applyAwarenessUpdate(this.awareness, fromB64(payload.awareness), this);
    });

    // Обработка запросов синхронизации от вновь подключившихся клиентов
    this.channel.on('broadcast', { event: 'sync-request' }, () => {
      const update = Y.encodeStateAsUpdate(this.doc);
      this.send('update', { update: toB64(update) });
      const awarenessUpdate = encodeAwarenessUpdate(
        this.awareness,
        Array.from(this.awareness.getStates().keys())
      );
      this.send('awareness', { awareness: toB64(awarenessUpdate) });
    });

    // Подписка на канал
    this.channel.subscribe((status) => {
      if (status === 'SUBSCRIBED') {
        this.isSubscribed = true;
        this.send('sync-request', {});
        if (onReady) onReady();
      }
    });

    // Вещание локальных изменений Yjs
```

```
    this.doc.on('update', (update: Uint8Array, origin: any) => {
      if (origin !== this) {
        this.send('update', { update: toB64(update) });
      }
    });
  }

  private send(event: string, payload: any) {
    if (this.isSubscribed) {
      this.channel.send({ type: 'broadcast', event, payload });
    }
  }

  public destroy() {
    this.channel.unsubscribe();
    this.awareness.destroy();
  }
}
```

3. Гибридный механизм сохранения: Анализ Concurrency & `FOR UPDATE` в PostgreSQL

Использование эемерных WebSocket-каналов (Supabase Realtime) обеспечивает минимальную задержку (Low Latency < 100ms) при пересылке изменений между браузерами. Однако эфирные каналы не гарантируют доставку и персистентность.

Когда AI-агент генерирует поток дельт и одновременно с этим несколько пользователей вносят правки, возникают параллельные запросы на сохранение состояния в БД. Если просто выполнять `UPDATE "Doc" SET yjsState = ...`, возникнет **состояние гонки (Race Condition)**: поздний запрос перезапишет ранний, приведя к разрушению CRDT-вектора и потере фрагментов документа.

Для решения этой проблемы реализована гибридная схема сохранения: клиент отправляет свои Yjs-дельты в БД через Server Actions, а БД выполняет бинарное слияние векторов под атомарной блокировкой строки `FOR UPDATE`:

```
// app/actions/doc.ts
'use server';

import * as Y from 'yjs';
import { prisma } from '@/lib/prisma';
import { Prisma } from '@prisma/client';
import { canAccessWorkspace } from '@/lib/workspace-access';

export async function saveDocContent(
  workspaceId: string,
  content: Prisma.InputJsonValue,
  yjsState?: string
) {
  if (!(await canAccessWorkspace(workspaceId))) {
    throw new Error('Workspace not found or access denied');
  }

  await prisma.$transaction(async (tx) => {
    if (yjsState) {
      const incoming = Buffer.from(yjsState, 'base64');
```

```
      // Блокировка строки документа FOR UPDATE на уровне PostgreSQL
      const rows = await tx.$queryRaw<{ yjsState: string | null }[]>`
        SELECT "yjsState" FROM "Doc"
        WHERE "workspaceId" = ${workspaceId}
        FOR UPDATE
      `;

      let merged = incoming;
      if (rows[0]?.yjsState) {
        const existing = Buffer.from(rows[0].yjsState, 'base64');
        // Атомарное слияние бинарных векторов Yjs
        merged = Buffer.from(Y.mergeUpdates([existing, incoming]));
      }

      await tx.doc.update({
        where: { workspaceId },
        data: {
          content,
          yjsState: merged.toString('base64'),
        },
      });
    } else {
      await tx.doc.update({
        where: { workspaceId },
        data: { content },
      });
    }
  });
}
```

Данная комбинация гарантирует, что даже при десятках параллельных фоновых сохранений от AI-агента и пользователей бинарные векторы Yjs объединяются транзакционно и безошибочно.

Надежное разрешение конфликтов на уровне данных создает устойчивый фундамент для построения графического слоя, где элементы холста требуют точного программного управления со стороны AI.

---

3. Визуальный движок tldraw и его экосистема

Интерактивный холст платформы Outpost построен на базе библиотеки `@tldraw/tldraw`. Выбор декларативного холста обусловлен необходимостью строгого программного управления графическими объектами: AI-агенту требуется детерминированная модель React DOM, где каждый элемент имеет четко определенные координаты (X,Y), геометрические размеры (ширина, высота), Z-индекс и точки привязки связей.

Архитектурный стек tldraw и разграничение Sync-механизмов

Стек визуализации холста состоит из двух уровней:

1. **Основное React-ядро (@tldraw/tldraw):** Векторный движок холста с бесконечным панорамированием, масштабированием и поддержкой пользовательских UI-фигур.
2. **Демонстрационный слой синхронизации (useSyncDemo):** В рамках текущей клиентской реализации вещание состояния Canvas-фигур и позиций пользовательских курсоров производится через хук

`useSyncDemo` из пакета `@tldraw/sync`, подключаемый к демо-инфраструктуре Cloudflare Workers (Durable Objects). В производстве для холста может применяться аналог `SupabaseYJSProvider`, разработанный для редактора документов.

Сравнительная матрица движков холста

Параметр сравнения	**tldraw**	**Excalidraw**	**Fabric.js**	**Konva.js**
Декларативный React/TS API	Нативный: Полная интеграция с React-компонентами и строгая типизация TypeScript.	Высокий: Поддержка React, но ограничение на внедрение сложносоставных UI-компонентов.	Низкий: Императивный Canvas API на базе JS-классов.	Средний: Использует react-konva обертку над 2D Context Canvas.
Кастомные UI-фигуры (Custom Shapes)	Да: Поддерживает внедрение полноценных HTML/React компонентов с собственным UI и логикой inside.	Ограничено: Фигуры формируются базовыми векторными примитивами.	Сложно: Требует ручной отрисовки через Canvas HTML5 Context.	Сложно: Требует верстки на Konva.Nodes.
Изоляция Z-индексов	Автоматическая: Полное управление слоями, фокусом и событиями мыши через встроенный стейт-менеджер.	Автоматическая: Ограниченная гибкость изоляции контекста сложных диалогов.	Ручная: Требует программной сортировки массива объектов.	Ручная: Управляется порядком узлов в Stage/Layer.
Встроенная синхронизация	Да (@tldraw/sync): Нативная поддержка трансляции дельт изменений объектов и курсоров.	Частично: Требуется отдельная настройка веб-сокетов или Firebase.	Отсутствует: Требует написания собственного слоя синхронизации.	Отсутствует: Требуется самостоятельная разработка обмена состояниями.

Проектирование и код кастомной фигуры ERD-таблицы

Для генерации схем баз данных движок tldraw расширен кастомной фигурой `EntityTableShapeUtil` (`components/entity-table-shape.ts`). Фигура поддерживает редактирование колонок, бейджи Primary Key (PK) / Foreign Key (FK), динамический расчет высоты и точную привязку стрелок связей к конкретным строкам:

```
// components/entity-table-shape.ts
import React from 'react';
import {
  BaseBoxShapeUtil,
  HTMLContainer,
  Rectangle2d,
  T,
  TLBaseShape
} from '@tldraw/tldraw';

export type IEntityRow = {
  id: string;
  name: string;
  type: string;
  isPK: boolean;
  isFK: boolean;
};

export type IEntityTableShape = TLBaseShape<
  'entityTable',
  {
    w: number;
    h: number;
    tableName: string;
    rows: IEntityRow[];
  }
>;

export const HEADER_HEIGHT = 40;
export const ROW_HEIGHT = 32;

export class EntityTableShapeUtil extends BaseBoxShapeUtil<IEntityTableShape> {
  static override type = 'entityTable' as const;

  static override props = {
    w: T.number,
    h: T.number,
    tableName: T.string,
    rows: T.Array(
      T.object({
        id: T.string,
        name: T.string,
        type: T.string,
        isPK: T.boolean,
        isFK: T.boolean,
      })
    ),
  };

  override getGeometry(shape: IEntityTableShape) {
    const calculatedHeight = HEADER_HEIGHT + shape.props.rows.length * ROW_HEIGHT;
    return new Rectangle2d({
      x: 0,
      y: 0,
      width: shape.props.w,
      height: Math.max(shape.props.h, calculatedHeight),
      isFilled: true,
```

```
    });
  }

  // Расчет координаты Y конкретной строки таблицы для точной привязки стрелок
  public getRowAnchorY(rowIndex: number): number {
    return HEADER_HEIGHT + rowIndex * ROW_HEIGHT + ROW_HEIGHT / 2;
  }

  override component(shape: IEntityTableShape) {
    return (
      <HTMLContainer style={{ pointerEvents: 'all' }}>
        <div className="bg-card border-border border rounded-md shadow-lg overflow-hidden
w-full h-full flex flex-col">
          {/* Шапка таблицы */}
          <div className="bg-primary/10 border-b border-border px-3 font-bold h-[40px]
flex items-center justify-between shrink-0">
            <span className="text-sm font-semibold truncate">{shape.props.tableName}
</span>
          </div>
          {/* Список полей */}
          <div className="divide-y divide-border/50 overflow-hidden">
            {shape.props.rows.map((row) => (
              <div
                key={row.id}
                className="px-3 h-[32px] flex items-center justify-between text-xs"
              >
                <div className="flex items-center gap-1.5 truncate">
                  {row.isPK && (
                    <span className="bg-amber-500/20 text-amber-500 px-1 rounded font-mono
text-[10px]">
                      PK
                    </span>
                  )}
                  {row.isFK && (
                    <span className="bg-blue-500/20 text-blue-500 px-1 rounded font-mono
text-[10px]">
                      FK
                    </span>
                  )}
                  <span className="font-medium truncate">{row.name}</span>
                </div>
                <span className="text-muted-foreground font-mono text-[11px] ml-2">
                  {row.type}
                </span>
              </div>
            ))}
          </div>
        </div>
      </HTMLContainer>
    );
  }
}
```

Слой отображения векторных фигур предоставляет интерфейс для программного манипулирования, оркестрация которого выполняется серверной и клиентской частью AI-движка.

4. Интеграция с Google Gemini и Vercel AI SDK

Интеграция искусственного интеллекта в платформе Outpost требует высокой скорости реакции и абсолютной предсказуемости результатов вызова инструментов (Tool Calling).

Серверная архитектура `/api/agent/route.ts`

Маршрут API построен на базе `Vercel AI SDK` (`streamText`) и модели `Google Gemini 1.5 Flash`.

Техническое обоснование выбора Google Gemini 1.5 Flash

1. **Скорость и экономичность:** Gemini 1.5 Flash обеспечивает миллисекундные задержки генерации ответа при минимальной стоимости токенов, что важно для потокового создания объектов на холсте.
2. **Нативный Tool Calling:** Высокая точность соблюдения Zod-схем при параллельном вызове множества инструментов.
3. **Режим рассуждений (thinkingConfig):** Поддержка передачи внутреннего потока мыслей модели (`includeThoughts: true`), что позволяет информировать пользователя о шагах агента до выполнения команд над DOM.
4. **Контекстное окно:** Включение объема до 1 млн токенов дает возможность передавать полные текстовые и графические снимки двух рабочих поверхностей без потери данных.

```
// app/api/agent/route.ts
import { google } from '@ai-sdk/google';
import { streamText, convertToCoreMessages } from 'ai';
import { checkUser } from '@/app/actions/check-user';
import { canAccessWorkspace } from '@/lib/workspace-access';
import { agentToolDefs } from '@/lib/agent-tools';

export async function POST(req: Request) {
  const user = await checkUser();
  if (!user) return new Response('Unauthorized', { status: 401 });

  const { messages, workspaceId, canvasContext, docContext } = await req.json();

  if (!(await canAccessWorkspace(workspaceId))) {
    return new Response('Workspace not found', { status: 403 });
  }

  // Динамическая инъекция текущего снимка состояния экранов в системный промпт
  const systemPrompt = `You are Outpost's workspace agent.
Current Canvas State: ${canvasContext || 'Not Open'}
Current Doc State: ${docContext || 'Not Open'}`;

  const result = await streamText({
    model: google('gemini-1.5-flash'),
    system: systemPrompt,
    messages: convertToCoreMessages(messages),
    tools: agentToolDefs,
    providerOptions: {
      google: {
        thinkingConfig: { includeThoughts: true },
      },
    },
  });
```

```
  return result.toDataStreamResponse({ sendReasoning: true });
}
```

Согласование состояний в Dual-Surface процессе (State Reconciliation)

На каждом шаге обращения к AI-агенту клиентский React-контекст собирает снимки состояния обоих рабочих экранов:

1. **Canvas Context (summarizeCanvas):** Обход дерева фигур `tldraw` с сериализацией сущностей в формат JSON (ID фигуры, координаты X/Y, типы полей ERD-таблиц, связи стрелок).
2. **Doc Context (summarizeDoc):** Считывание текста или JSON AST редактора `BlockNote`.

Полученные данные объединяются и внедряются в системный промпт (`Current Canvas State`, `Current Doc State`). Это дает агенту точное понимание расположения объектов, позволяя изменять существующие фигуры, а не создавать дубликаты.

Архитектура Client-Side Tool Execution

Ключевым архитектурным решением платформы является **исполнение инструментов на стороне клиента (Client-Side Tool Execution)**.

Почему вызов инструментов вынесен на клиент, а не исполняется на сервере?

Экземпляры графического редактора `tldraw` (`editor`) и редактора документов `BlockNoteEditor` существуют исключительно в памяти браузера конкретного пользователя внутри React DOM-дерева. Серверный API-роут `/api/agent/route.ts` не имеет прямого доступа к эти объектам.



Механика Client-Side выполнения:

1. **Генерация спецификации:** LLM возвращает событие вызова инструмента (например, `createEntityTable` с аргументами).
2. **Перехват в React (useWorkspaceAgentChat):** Vercel AI SDK передает вызов на клиент в обработчик `onToolCall`.
3. **Прямая манипуляция:** Клиентский код вызывает соответствующие методы редакторов (`editor.createShape(...)` для tldraw или `editor.insertBlocks(...)` для BlockNote).
4. **Возврат статуса:** Результат выполнения возвращается в поток сообщений через `chat.addToolOutput()`. Параметр `sendAutomaticallyWhen` возобновляет потоковую генерацию ответа языковой моделью.

Результаты выполнения инструментов фиксируются в хранилище проектов, управляемом Kanban-доской.

---

5. Архитектура Kanban-доски и алгоритмы пространственного упорядочивания

Для визуализации и структурирования рабочих пространств (Workspaces) в платформе Outpost реализована интерактивная Kanban-доска. Управление состояниями карточек распределено по четырем колонкам статусов: `PLANNING`, `IN_PROGRESS`, `IN_REVIEW` и `DONE`.

Компоновка Kanban-доски

Интерфейс доски реализован с использованием библиотеки `@hello-pangea/dnd` (форк `react-beautiful-dnd`). Иерархия компонентов включает:

- `DragDropContext`: Глобальный контекст, обрабатывающий завершение перетаскивания (`onDragEnd`).
- `Droppable`: Колонки статусов, выступающие в роли зон сброса.
- `Draggable`: Интерактивные карточки рабочих пространств.

Алгоритм дробного индексирования (Fractional Indexing)

Традиционный подход к хранению порядка элементов в СУБД с использованием целочисленных индексов (`position: 1, 2, 3...`) требует проведения массовых тяжелых SQL-операций `UPDATE` для всех соседних элементов при перемещении одной карточки в начало или середину списка (O(N) операций).

В Outpost применен алгоритм **дробного индексирования (Fractional Indexing)**. Каждая карточка содержит числовую позицию типа `Float` (`Double Precision` в PostgreSQL). При перемещении элементы пересчитываются локально без затрагивания остальной коллекции (O(1) операций).

Математическая модель `positionBetween`:

```
// lib/utils.ts
export function positionBetween(before: number | null, after: number | null): number {
  // 1. Вставка между двумя карточками: берется среднее арифметическое
  if (before !== null && after !== null) {
    return (before + after) / 2;
  }
  // 2. Вставка в конец колонки: позиция последнего элемента + 1
  if (before !== null) {
    return before + 1;
  }
  // 3. Вставка в начало колонки: позиция первого элемента - 1
```

```
  if (after !== null) {
    return after - 1;
  }
  // 4. Единственная карточка в колонке
  return 0;
}
```

При перемещении элементов обновляется **строго одна запись** в базе данных, исключая блокировку таблицы.

Разбор Server Actions управления пространствами

Управление карточками выполняется через защищенные Server Actions (`app/actions/workspace.ts`):

1. Перемещение карточки `moveWorkspace`

```
// app/actions/workspace.ts
'use server';

import { prisma } from '@/lib/prisma';
import { canAccessWorkspace } from '@/lib/workspace-access';
import { WorkspaceStatus } from '@prisma/client';

export async function moveWorkspace(
  workspaceId: string,
  status: WorkspaceStatus,
  position: number
) {
  if (!(await canAccessWorkspace(workspaceId))) {
    throw new Error('Workspace not found or access denied');
  }

  const updated = await prisma.workspace.update({
    where: { id: workspaceId },
    data: { status, position },
  });

  return updated;
}
```

2. Безопасное удаление `deleteWorkspace`

```
// app/actions/workspace.ts
'use server';

import { prisma } from '@/lib/prisma';
import { auth } from '@clerk/nextjs/server';

export async function deleteWorkspace(workspaceId: string) {
  const { userId, orgId } = await auth();
  if (!userId) throw new Error('Unauthorized');

  const workspace = await prisma.workspace.findUniqueOrThrow({
    where: { id: workspaceId },
    select: { creatorId: true, clerkOrgId: true },
```

```
  });

  const isCreator = workspace.creatorId === userId;
  const isOrgMatch = workspace.clerkOrgId && workspace.clerkOrgId === orgId;

  if (!isCreator && !isOrgMatch) {
    throw new Error('Only creator or org admin can delete workspace');
  }

  await prisma.workspace.delete({
    where: { id: workspaceId },
  });
}
```

Алгоритмы управления графическими интерфейсами тесно переплетены с директивами промпт-инжиниринга, задающими поведение AI-агента.

---

6. Промпт-инжиниринг и специализированные инструкции

Промпт-инжиниринг в платформе Outpost является базовым кодом управления AI-агентом. Системные промпты определяют способность агента правильно выбирать рабочую поверхность, интерпретировать текущие снимки состояния и генерировать корректные вызовы инструментов.

Системный промпт AI-агента (`app/api/agent/route.ts`)

Ниже представлен полный текст системного промпта, определяющего правила работы агента над холстом и документом:

```
You are Outpost's workspace agent. A workspace has two surfaces: a canvas and a doc.
You change them ONLY by calling tools, never by describing changes in text.
Never use em-dashes (—).

CHOOSING WHERE TO WORK:
- If the user names a surface ("in the doc", "on the canvas"), edit ONLY that surface.
- "Write a spec for this diagram" -> Read canvas context, write to doc.
- "Build a database schema" -> Build ERD table shapes on canvas, document in doc.

CANVAS RULES:
- Use entityTable shape for database schemas. Always set valid position coordinates (x, y)
without overlapping.
- When creating arrows between shapes, use createArrowBetween with valid shape IDs.
- Keep layout clean: top-to-bottom or left-to-right flow with 80px minimum spacing.

DOC RULES:
- Write clean Markdown blocks. Prefer targeted edits over full document replaces.
- Use headers (#, ##), bullet lists, and bold text for specs.
```

Попоследовательные промпты для верстки Landing Page

Верстка конверсионной главной страницы платформы строилась поэтапно путем генерации специализированных инструкций для AI-инструмента разработки (Claude Code / Copilot):

Этап 1: Hero-секция и видеоконтейнер

```
Build the landing page hero section in app/page.tsx using our 'topo-bg' utility class on
dark background.
Add a small mono badge 'EARLY ACCESS' on top with a diamond marker.
Headline: 'An AI agent that draws and writes alongside you and your team live'.
Subheadline and two action buttons: 'Start building free' (redirects to signin) and 'See
how it works' (scrolls down).
Under hero text, add the product video container loading public/videos/demo.mp4 (autoplay,
muted, loop, playsinline) inside a rounded card with 'surface-frame' corner marks and
'scroll-tilt' CSS effect.
```

Этап 2: Вынос данных и лента аудитории

```
Refactor landing page copy into a data file at data/landing.ts.
Below the video container, add a short audience strip showing who the platform is for
('For Product Teams', 'For Founders', 'For Solo Builders') with clean badges and muted
divider lines.
```

Этап 3: Карточки ключевых возможностей

```
Create a feature section with three main surface rows:
1. Infinite Canvas: agent drawing flowcharts and ERD tables live.
2. BlockNote Docs: Notion-like rich text editor with live Yjs collaboration.
3. Team Board: Drag and drop Kanban board managing workspaces across organizations.
Each row should contain an icon, title, detailed description, and a static UI mock
component placed next to it.
```

Этап 4: Секция «How It Works»

```
Add a 3-step 'How It Works' section below features:
Step 1: 'Prompt or Speak' - type or use speech input to state requirements.
Step 2: 'Watch It Build' - agent draws shapes on canvas and writes doc blocks in real time
with live thinking.
Step 3: 'Collaborate & Export' - invite team members via Clerk orgs or export work to PDF.
```

Этап 5: Табы тарификации и футер

```
Integrate a Pricing section using Shadcn UI Tabs switching between 'Personal' and
'Organization' plans.
Embed Clerk's <PricingTable /> component inside tab contents.
Finish with a closing CTA section using 'topo-bg' background and a simple footer with link
badges and text 'Made with love by roadsidecoder'.
```

Промпт для Claude Code / Opus (Кастомная ERD-фигура)

Для создания сложного компонента `EntityTableShapeUtil` с поддержкой снайпинга стрелок связей использовался промпт:

```
Following tldraw's custom shapes documentation, add an 'entityTable' shape to our canvas
in components/entity-table-shape.ts.
Shape properties:
- tableName: string
- rows: Array of { id, name, type, isPK, isFK }
Height rules:
- Height dynamically follows row count: HEADER_HEIGHT (40px) + rows.length * ROW_HEIGHT
(32px).
- Width is manually resizable.
Visuals:
- Styled with Tailwind using shadcn dark theme CSS variables.
- Show PK (amber badge) and FK (blue badge) tags next to row names.
Binding logic:
- Implement getRowAnchorY(rowIndex) calculation so arrows drawn onto the table
automatically snap to the nearest specific row center line instead of the bounding box
center.
- Export shape util and register in CanvasEditor components prop.
```

Промпт-инжиниринг платформы напрямую интегрирован с подсистемой авторизации, мультиарендности и тарификации.

---

7. Clerk: B2B Мультиарендность, авторизация и монетизация

Коммерческая эксплуатация платформы Outpost основана на платформе **Clerk**, обеспечивающей сессии пользователей, B2B-мультиарендность и тарификацию подписок (Clerk Billing).

Мультиарендность (Personal vs Organization Workspaces)

В платформе активирован режим `Membership optional`. Пользователь может работать индивидуально (Personal Workspace) или переключаться между организациями (Organization Workspaces).

Изоляция данных в БД:

- **Личное пространство:** Записи содержат `clerkOrgId = null` и привязаны к `creatorId = userId`.
- **Пространство организации:** Записи содержат заполненный `clerkOrgId = orgId`. Доступ предоставляется всем членам организации.

Безопасность и управление доступом

1. Определение области запроса `scopeWhere`

```
// lib/workspace-scope.ts
import { Prisma } from '@prisma/client';

export function scopeWhere(userId: string, orgId?: string | null):
Prisma.WorkspaceWhereInput {
  if (orgId) {
    return { clerkOrgId: orgId };
  }
  return {
    creatorId: userId,
```

```
    clerkOrgId: null
  };
}
```

2. Проверка прав доступа `canAccessWorkspace`

```
// lib/workspace-access.ts
import { auth, clerkClient } from '@clerk/nextjs/server';
import { prisma } from '@/lib/prisma';

export async function canAccessWorkspace(workspaceId: string): Promise<boolean> {
  const { userId, orgId } = await auth();
  if (!userId) return false;

  const workspace = await prisma.workspace.findUnique({
    where: { id: workspaceId },
    select: { creatorId: true, clerkOrgId: true },
  });

  if (!workspace) return false;

  if (!workspace.clerkOrgId) {
    return workspace.creatorId === userId;
  }

  if (workspace.clerkOrgId === orgId) return true;

  const client = await clerkClient();
  const memberships = await client.users.getOrganizationMembershipList({ userId });
  return memberships.data.some((m) => m.organization.id === workspace.clerkOrgId);
}
```

3. Защита маршрутов через Proxy Middleware

```
// proxy.ts
import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server';

const isProtectedRoute = createRouteMatcher([
  '/dashboard(.*)',
  '/workspace(.*)',
]);

export default clerkMiddleware(async (auth, req) => {
  if (isProtectedRoute(req)) {
    await auth.protect();
  }
});

export const config = {
  matcher: ['/((?!.*\\..*|_next).*)', '/', '/(api|trpc)(.*)'],
};
```

Интеграция Clerk Billing (Seat-Based & Quotas)

Монетизация платформы включает тарифные планы с ограничениями по рабочим местам (Seats) и месячным квотам AI-промптов.

Сетка тарифных планов:

1. **Free Plan:** Лимит 3 рабочих пространства, 20 AI-промптов/мес.
2. **Pro Plan ($12/мес):** Безлимитные пространства, PDF-экспорт, 500 AI-промптов/мес.
3. **Team Plan ($12/seat/мес):** Seat-based тарификация для организаций. Квота: 500 промптов/seat (например, 4 участника = 2000 промптов/мес на команду).
4. **Enterprise Plan ($25/seat/мес):** Покресельная тарификация, 2000 промптов/seat.

Подсчет использования и перехват лимитов

Перед исполнением каждого промпта сервер рассчитывает права и текущий расход:

```
// lib/billing.ts
import { prisma } from '@/lib/prisma';
import { getEntitlements } from '@/lib/entitlements';

export async function getPromptUsage(ownerId: string) {
  const startOfMonth = new Date();
  startOfMonth.setUTCDate(1);
  startOfMonth.setUTCHours(0, 0, 0, 0);

  const used = await prisma.agentPrompt.count({
    where: {
      ownerId,
      createdAt: { gte: startOfMonth },
    },
  });

  const entitlements = await getEntitlements(ownerId);
  return {
    used,
    limit: entitlements.promptLimit,
    isPaid: entitlements.isPaid,
  };
}

export async function recordPrompt(ownerId: string, userId: string, workspaceId: string) {
  await prisma.agentPrompt.create({
    data: { ownerId, userId, workspaceId },
  });
}
```

Если лимит исчерпан (`used >= limit`), сервер возвращает ошибку `LIMIT_REACHED` . Перехватчик на клиенте выносит диалоговое окно `UpgradeDialog` :

```
// hooks/use-workspace-agent-chat.ts
if (error?.message?.includes('LIMIT_REACHED')) {
  openUpgradeModal({
    title: 'Превышен лимит AI-промптов',
    description: 'Вы исчерпали доступные промпты на этот месяц. Перейдите на план Pro или
```

```
Team для продолжения работы.',
  });
}
```

Диалоговое окно инкапсулирует компонент `<PricingTable />` из библиотеки `@clerk/nextjs`, позволяя совершить мгновенный апгрейд подписки без покидания рабочего пространства.

Заключение

Архитектура платформы **Outpost** представляет собой полностью законченный и масштабируемый паттерн для разработки B2B Agentic SaaS-приложений. Интеграция графического векторного движка `tldraw`, блочного редактора `BlockNote`, конфликтно-устойчивых алгоритмов `Yjs (CRDT)`, оркестрации `Vercel AI SDK` с моделью `Google Gemini 1.5 Flash` и безопасного B2B-контура `Clerk + Supabase` образует технологическую экосистему, готовую к продуктовой эксплуатации под высокой нагрузкой.